VERIQTA

KUBERNETES

IN A NUTSHELL

THE COMPLETE KUBERNETES MENTAL MODEL
FOR ENGINEERS & ARCHITECTS

CONCEPTS | ARCHITECTURE | COMMANDS | BEST PRACTICES | TROUBLESHOOTING

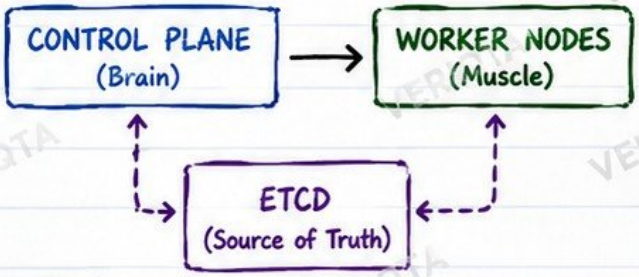
BUILD • OPERATE • SCALE • SECURE

1. KUBERNETES ARCHITECTURE IN A NUTSHELL

THE BIG PICTURE

- Kubernetes is a CONTROL SYSTEM.
- It drives the actual state to match the desired state.
- It is NOT just a container orchestrator.
- It continuously observes, decides and acts.

HIGH LEVEL VIEW



CONTROL PLANE (BRAIN)

API SERVER
Front door. All requests go through here.

ETCD
Highly consistent key-value store. Source of truth.

SCHEDULER
Decides which node runs new Pods.

CONTROLLER MANAGER
Runs controllers that move the system toward desired state (e.g., Deployment, Node, Endpoints).

WORKER NODE (MUSCLE)

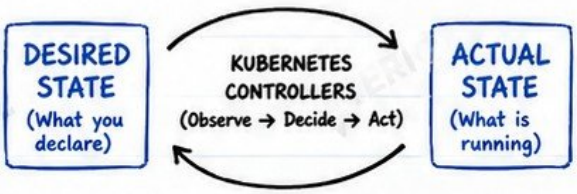
KUBELET
Agent that ensures containers are running in a Pod.

KUBE-PROXY
Handles Service networking and load balancing.

CONTAINER RUNTIME
(containerd / CRI-O)
Pulls images and runs containers.

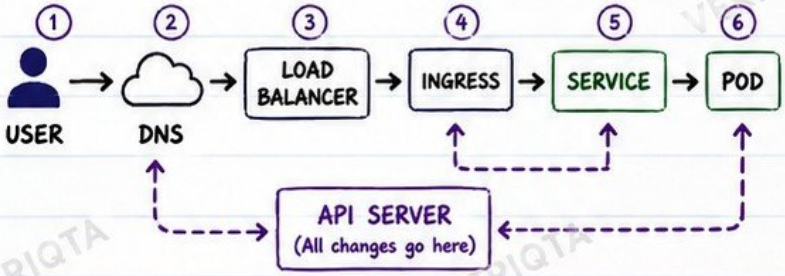
Runs Your Workloads

DESIRED STATE & RECONCILIATION



Kubernetes continuously closes the gap until Actual State = Desired State.

END-TO-END REQUEST FLOW



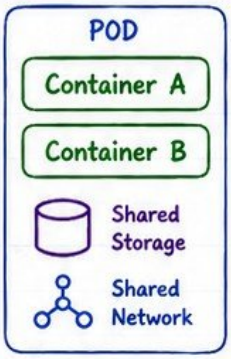
PRODUCTION NOTE
Kubernetes is a CONTROL SYSTEM, not simply a container orchestrator. Its job is to keep your system in the state you declare.

Think in systems, not static infrastructure.

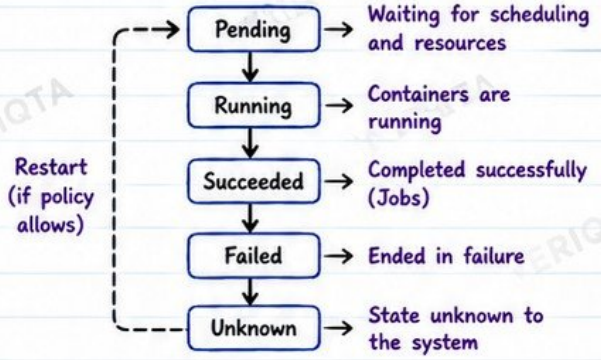
2. PODS IN A NUTSHELL

WHAT IS A POD?

- Smallest deployable unit in Kubernetes.
- One or more containers that share the same network, storage and lifecycle.
- Pods are ephemeral and replaced, not pets.



POD LIFECYCLE

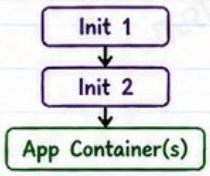


POD PHASES vs CONDITIONS

PHASE	MEANING	COMMON CONDITIONS
Pending	Pod accepted but not running yet.	PodScheduled=False, Initialized=False
Running	At least one container is running.	Ready=True/False, ContainersReady
Succeeded	All containers completed successfully.	Completed=True
Failed	One or more containers terminated in failure.	Failed=True
Unknown	Kubelet can't report status.	Ready=Unknown

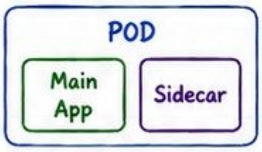
INIT CONTAINERS

1. Run before app containers.
2. Must succeed for Pod to start.
3. Common for setup, migrations, or waiting on dependencies.

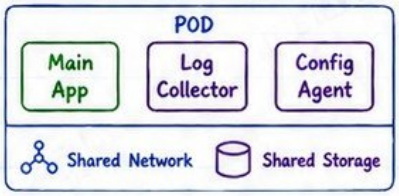


SIDECARS

- Run alongside main containers.
- Add supporting capabilities (logging, proxy, monitoring).
- Share network and storage.



MULTI-CONTAINER POD



Use when containers need to:

- Share data
- Communicate locally
- Work tightly together

RESTART POLICIES

POLICY	BEHAVIOR
Always	Restart containers if they fail (default for Deployments).
OnFailure	Restart only if container exits with an error.
Never	Do not restart (for Jobs that must not repeat).

WHY PODS ARE DISPOSABLE

- Controllers (Deployments, StatefulSets) replace failed Pods automatically.
- Pods are cattle, not pets.
- Designed for failure: Kubernetes heals by recreating Pods.
- Do not store important data in a Pod filesystem. Use Persistent Volumes.

COMMON DEBUGGING STATES

Pending	• Unschedulable, resources, or node issues.	kubectl describe pod <pod> kubectl get events
CrashLoopBackOff	• Container starts and keeps crashing.	kubectl logs <pod> kubectl describe pod <pod>
ImagePullBackOff	• Image not found or pull failed.	kubectl describe pod <pod> Check image name & access
OOMKilled	• Container killed due to memory limit.	kubectl describe pod <pod> Check limits & usage

PRODUCTION NOTE

Design for failure. Let Kubernetes replace Pods. Build idempotent, stateless apps and keep important data outside the Pod.

SENIOR TIP

When in doubt:
DESCRIBE → LOGS → EVENTS → METRICS → FIX ROOT CAUSE

3. DEPLOYMENTS AND REPLICASETS IN A NUTSHELL

THE RELATIONSHIP

DEPLOYMENT
Manages desired state and rollouts

→

REPLICASET
Ensures the correct number of Pods

→

POD
Runs your application

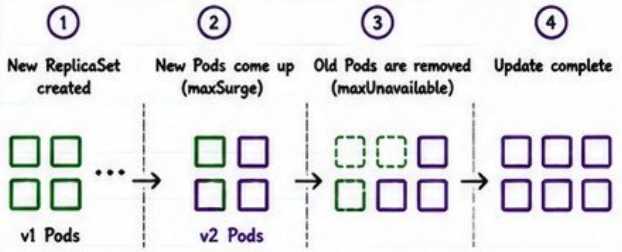
DESIRED REPLICAS

- `.spec.replicas` defines how many Pods should be running.
- ReplicaSet continuously compares actual vs desired state.
- If a Pod dies, a new one is created.
- Kubernetes converges to the desired state.

Always working to match the desired state

ROLLING UPDATE STRATEGY

Default Strategy: RollingUpdate



v1 = Old Version v2 = New Version

maxSurge AND maxUnavailable

SETTING	WHAT IT DOES
maxSurge	How many extra Pods can be created above the desired replicas during update. → Extra Pods allowed
maxUnavailable	How many Pods can be taken down below the desired replicas during update. → Pods that can be down

EXAMPLE:
- Desired Replicas = 5
- maxSurge = 1 → Up to 6 Pods allowed
- maxUnavailable = 1 → Minimum 4 Pods must stay running

ROLLOUT HISTORY

- Every change creates a new ReplicaSet.
- Kubernetes keeps a history of deployments.
- View history: `kubectl rollout history deployment/<name>`
- View details: `kubectl rollout history deployment/<name> --revision=N`

ROLLBACK

- Roll back to a previous revision.
- Command: `kubectl rollout undo deployment/<name>`
- Kubernetes scales down the new ReplicaSet and scales up the old one.

PRODUCTION FAILURE: HEALTHY ROLLOUT, UNHEALTHY APPLICATION

- Rollout shows "success" but users still have issues.
- Root causes are usually in: code, config, dependencies, databases, external services, or bad traffic routing.
- Always validate with real checks, not just rollout status.

CHECK BEYOND THE ROLLOUT
- ✓ Logs and metrics
- ✓ Readiness / Liveness
- ✓ Business endpoints
- ✓ User experience

Rollout success ≠ Application success

SENIOR TIP
Use small, frequent rollouts. Test in production, then roll forward.

USEFUL COMMANDS

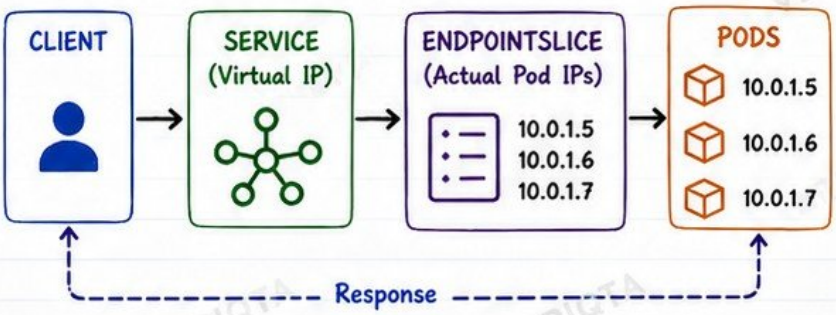
`kubectl get deploy`	List Deployments	`kubectl rollout status deploy/<name>`	Check rollout progress
`kubectl get rs`	List ReplicaSets	`kubectl rollout history deploy/<name>`	View rollout history
`kubectl get pods`	List Pods	`kubectl rollout undo deploy/<name>`	Rollback to previous revision

4. SERVICES AND KUBERNETES NETWORKING

SERVICE TYPES

CLUSTERIP (Default)	Internal only. Accessible within the cluster.
NODEPORT	Exposes service on each node IP at a static port.
LOADBALANCER	Exposes service externally via cloud provider Load Balancer.
HEADLESS SERVICE	No ClusterIP. Returns Pod IPs directly. Used for StatefulSets and custom discovery.

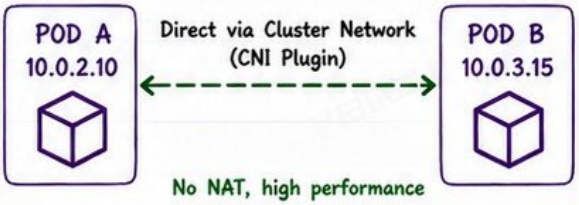
HOW A REQUEST REACHES A POD



SERVICE → ENDPOINTSLICE → POD

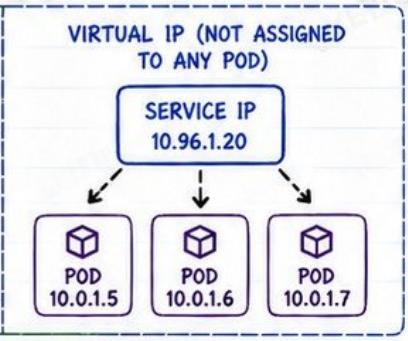
- Watches Services and EndpointSlices.
- Programs rules on each node (iptables / IPVS).
- Forwards traffic from Service IP or NodePort to the correct Pod.
- Maintains load balancing across Pods.

POD-TO-POD COMMUNICATION



WHY SERVICE IPS ARE VIRTUAL

- Service IPs are not assigned to any network interface.
- They exist only in Kubernetes data plane.
- Traffic to the Service IP is intercepted and load balanced to real Pod IPs.
- This allows Pods to be replaced without changing the Service IP.



SUMMARY TABLE

TYPE	EXPOSED TO	IP	USE CASE
ClusterIP	Inside cluster	Internal virtual IP	Default. Internal apps
NodePort	Outside via Node IP:Port	Node IP + Static Port	Dev / Testing / Simple exposure
LoadBalancer	Outside via Cloud LB	Cloud LB IP	Production external access
Headless	Inside cluster	No ClusterIP (DNS only)	StatefulSets / Custom discovery

PRODUCTION NOTE

Services provide stable endpoints for dynamic Pods. Never call Pods directly from outside the cluster. Always use Services.

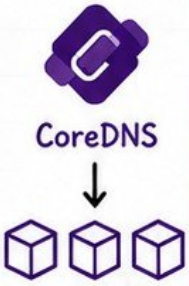
SENIOR TIP

Think: Service for stability, Labels for selection, EndpointSlice for reality, kube-proxy for delivery.

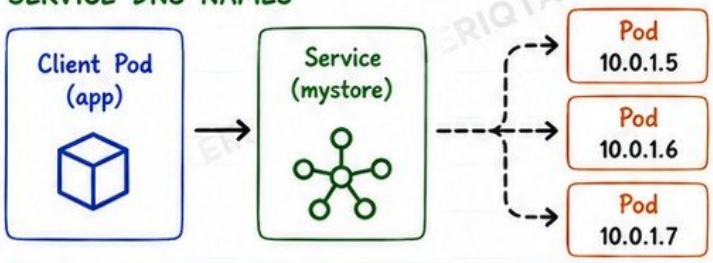
5. DNS AND SERVICE DISCOVERY

COREDNS: THE CLUSTER DNS

- CoreDNS is the DNS server for your cluster.
- Runs as Pods in kube-system namespace.
- Watches Services and Pods.
- Answers DNS queries from inside the cluster.



SERVICE DNS NAMES



DNS NAME FORMAT

<service>.<namespace>.svc.cluster.local

Example: mystore.default.svc.cluster.local

NAMESPACE-AWARE RESOLUTION

FROM POD IN	TYPES SERVICE AS	RESOLVES TO
Same Namespace	mystore	mystore.<ns>.svc.cluster.local
Other Namespace	mystore.other-ns	mystore.other-ns.svc.cluster.local
Fully Qualified	mystore.other-ns.svc.cluster.local	Exact service

Short names only work within the same namespace.

POD DNS

- Each Pod gets its own DNS A record.
- Format: <pod-ip>.<namespace>.pod.cluster.local
- Mostly used by the system.
- Pods should use Services, not Pod IPs.

SEARCH DOMAINS

/etc/resolv.conf

Pods get a list of search domains used when you type short names.

- <namespace>.svc.cluster.local
- svc.cluster.local
- cluster.local

NDOTS (DEFAULT: 5)

- ndots defines how many dots a name must have to be treated as FQDN.
- If dots < ndots → Kubernetes appends search domains.
- If dots ≥ ndots → Query is sent as-is.
- High ndots can cause extra DNS queries and latency.

TIP

Use FQDN in high-latency environments or reduce ndots if needed.

DEBUGGING DNS FROM INSIDE A POD

1. `nslookup mystore.default.svc.cluster.local` — Check DNS resolution.
2. `dig mystore.default.svc.cluster.local` — Detailed DNS query.
3. `kubectl exec -it <pod> -- cat /etc/resolv.conf` — View search domains and nameserver.
4. `kubectl exec -it <pod> -- ping mystore` — Test DNS + connectivity.
5. `kubectl exec -it <pod> -- nslookup kubernetes.default` — Test cluster DNS.

PRODUCTION FAILURE

DNS latency = Application latency

- Slow DNS causes: timeouts, retries, and cascading failures.
- Root causes: CoreDNS overload, bad upstream DNS, high ndots, network issues.
- Monitor CoreDNS, use caching, and keep queries fast and local.

SENIOR TIP

Always prefer Services for discovery.

Avoid hardcoding Pod IPs.

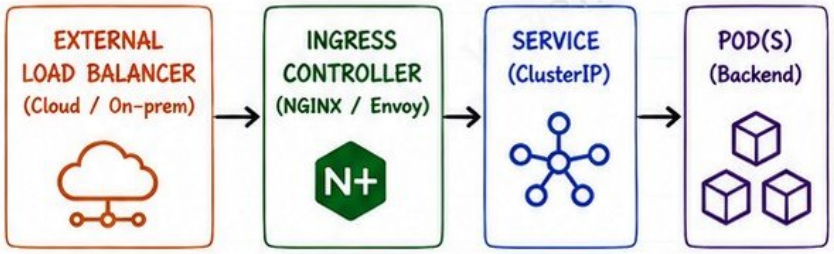
Design for change.

6. INGRESS AND EXTERNAL TRAFFIC

KEY BUILDING BLOCKS

INGRESS RESOURCE	Defines rules for how external traffic reaches Services.
INGRESS CONTROLLER	Watches Ingress resources and programs the proxy.
GATEWAY API	Next-gen API for traffic management. More powerful and flexible.

EXTERNAL TRAFFIC FLOW



Ingress Controller is the smart traffic proxy.
It inspects requests and routes to the right Service.

HOST AND PATH ROUTING

TYPE	EXAMPLE	ROUTES TO
Host Based Routing	Host: app.example.com	Service: app-service
Path Based Routing	app.example.com/api	Service: api-service
	app.example.com/	Service: web-service
Host + Path Routing	api.example.com/v1	Service: v1-service

More specific path wins.
Exact path (/api/v1) > Prefix path (/api) > Root (/)

TLS TERMINATION

- TLS terminates at the Ingress Controller.
- Traffic to Services inside the cluster is usually HTTP.
- Store certs in Kubernetes Secrets.
- Supports SNI for multiple hosts.

GATEWAY API OVERVIEW

GATEWAY	Listens on the network (LB / IP / Port)
HTTP ROUTE	Rules for HTTP traffic (host, path, headers)
BACKEND	References Services
ATTACHMENT	Links everything together

FUTURE STANDARD
More power, better control, stronger extensibility.

COMMON HTTP RESPONSE CODES

CODE	MEANING	COMMON CAUSES
404	Not Found	Wrong host/path, rule missing, backend not configured.
502	Bad Gateway	Service/Pods unhealthy, connection refused, upstream timeout.
503	Service Unavailable	No healthy endpoints, maintenance, or controller not ready.

DEBUGGING 404, 502, 503

404 NOT FOUND
- Check Ingress rules (host / path)
- kubectl get ingress
- kubectl describe ingress
- Confirm Service exists
- Confirm endpoints exist

502 BAD GATEWAY
- Check Service endpoints
- kubectl get endpoints
- kubectl describe svc
- Check Pod logs
- Check readiness probes
- Test from inside cluster

503 SERVICE UNAVAILABLE
- No ready endpoints
- kubectl get endpoints
- Check Pod status
- Check readiness probes
- Scaling or rollout issues?

GENERAL CHECKS
- kubectl get pods -A
- kubectl get svc -A
- kubectl get ingress -A
- kubectl logs -n <ns> <ingress-controller-pod>
- Check DNS resolution

PRODUCTION NOTE
Ingress shows "success" but your app can still fail.
Always test end-to-end: DNS → LB → Ingress → Service → Pod → App.

SENIOR TIP
Monitor Ingress and backend health.
Design for failure: probes, timeouts, retries, and clear error handling.

7. CONFIGMAPS, SECRETS AND APPLICATION CONFIGURATION

CONFIGMAP vs SECRET

CONFIGMAP	SECRET
• Non-sensitive data	• Sensitive data
• Store as plain text	• Stored as base64 (encoded)
• Examples: app.properties, feature flags, URLs	• Examples: passwords, tokens, SSL certs
• Can be viewed easily.	• Restricted access

Rule: If it's sensitive, use SECRET.
Everything else, use CONFIGMAP.

ENV VARS vs MOUNTED VOLUMES

ENVIRONMENT VARIABLES

- Injected into container env
- Good for small values
- Visible in 'env' and process list
- Not ideal for large or structured data

valueFrom:
configMapKeyRef / secretKeyRef

MOUNTED VOLUMES

- Files mounted into Pod
- Good for large configs
- Better for certs, keys, JSON, YAML, scripts
- More secure (not in env)
- Supports updates

volume:
configMap / secret

SECRET HANDLING

- Secrets are stored in etcd (base64 encoded).
- Limit access with RBAC.
- Never log or print secrets.
- Avoid putting secrets in container images.
- Use ServiceAccounts, not static credentials.
- Use short-lived tokens when possible.

Base64 is NOT encryption.
Anyone with access can decode it.

CONFIGURATION UPDATES

ConfigMap / Secret Updated → Mounted as Volume → App Reads New Data

Env vars are NOT updated automatically.
You must restart the Pod to pick up changes.

IMMUTABLE CONFIGURATION

Mark ConfigMaps/Secrets as immutable to prevent accidental changes.
Improves safety and stability.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  immutable: true
data:
  app.properties: |
  log.level=info
```

Locked and protected.

EXTERNAL SECRET SYSTEMS

EXTERNAL STORES
- AWS Secrets Manager
- HashiCorp Vault
- Azure Key Vault
- Google Secret Manager

Sync / Fetch →

KUBERNETES CLUSTER
External Secrets Operator
Secrets | ConfigMaps

Keeps secrets out of Git and reduces manual work.

SECURITY REMINDER

Base64 is ENCODING, not ENCRYPTION.
Anyone with access to the cluster can decode it.
Protect access. Limit permissions. Audit regularly.

BASE64 → DECODE → PLAIN TEXT

QUICK REFERENCE

- Use ConfigMap for non-sensitive configuration.
- Use Secret for sensitive information.
- Use env vars for small values.
- Use volumes for large files and certificates.
- Restart Pod if using env vars after update.

8. REQUESTS, LIMITS AND KUBERNETES QOS

RESOURCE BASICS

CPU REQUESTS	• Minimum CPU guaranteed by the scheduler.
MEMORY REQUESTS	• Minimum memory guaranteed by the scheduler.
CPU LIMITS	• Maximum CPU a container can use.
MEMORY LIMITS	• Maximum memory a container can use.

EXAMPLE: CONTAINER RESOURCES

```
resources:
  requests:
    cpu: "250m"        # 0.25 CPU
    memory: "256Mi"    # 256 MiB
  limits:
    cpu: "1000m"       # 1 CPU
    memory: "512Mi"    # 512 MiB
```

QUALITY OF SERVICE (QOS) CLASSES

CLASS	CRITERIA	CPU GUARANTEE	MEMORY GUARANTEE	SCHEDULING	EVICTION PRIORITY
GUARANTEED	CPU request = limit AND Memory request = limit	Guaranteed	Guaranteed	Highest priority for scheduling	Last to be evicted
BURSTABLE	Requests < Limits (for CPU or memory)	Partial	Partial	Medium priority for scheduling	May be evicted
BESTEFFORT	No requests No limits	None	None	Lowest priority for scheduling	First to be evicted

CPU THROTTLING

- When CPU usage hits the limit, the container is throttled.
- The process is paused and resumes later.
- This causes high latency and slow responses.
- Check with: `kubectl top pod`

OOMKILLED (OUT OF MEMORY)

- When memory usage hits the limit, the container is killed.
- The process exits immediately.
- Pod restarts (CrashLoopBackOff).
- Check with: `kubectl describe pod <pod>`

HOW REQUESTS AFFECT SCHEDULER DECISIONS

Pod Spec (with requests) → Scheduler checks node capacity → Finds node with enough free requests → Pod is scheduled to that node

Limits are NOT used for scheduling. Only requests are considered.

SENIOR ENGINEER TIP

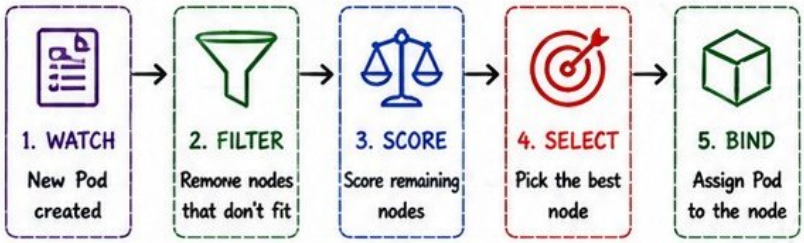
Requests influence scheduling.
Limits influence runtime behavior.
Set realistic requests for stability and efficient packing of nodes.

KEY TAKEAWAYS

- Set requests to get scheduled.
- Set limits to control usage.
- Right sizing prevents throttling, OOMKills and evictions.

9. SCHEDULING IN A NUTSHELL

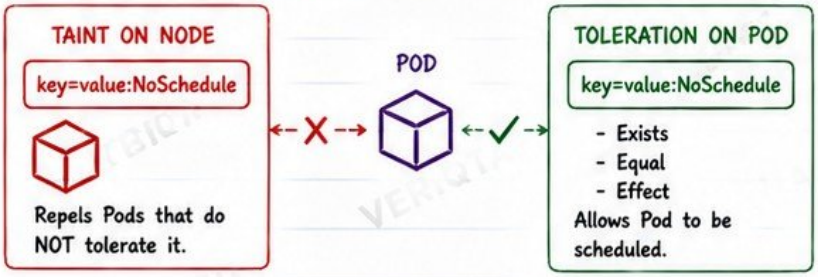
SCHEDULER WORKFLOW



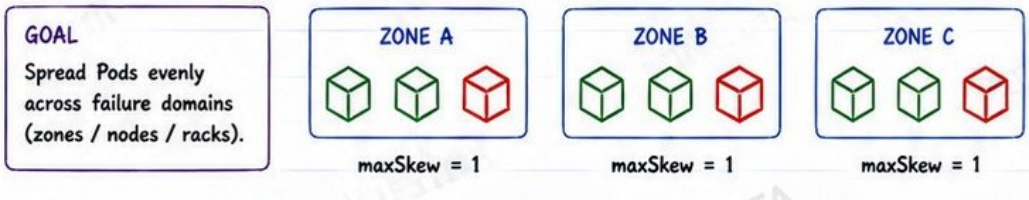
NODE SELECTION METHODS

nodeSelector	Simple key=value match (hard requirement)
Node Affinity	Rules based on labels (required / preferred)
Pod Affinity	Schedule near Pods that match rules
Pod Anti-Affinity	Avoid Nodes with matching Pods
Taints & Tolerations	Repel Pods unless tolerated
topologySpread-Constraints	Spread Pods evenly across zones / nodes / racks

TAINTS AND TOLERATIONS



TOPOLOGY SPREAD CONSTRAINTS (EXAMPLE)



KEY FIELDS

- maxSkew
- topologyKey
- whenUnsatisfiable
  DoNotSchedule
  or
  ScheduleAnyway

UNSCHEDULABLE PODS: COMMON REASONS

- ⊗ Not enough CPU / Memory (requests too high).
- ⊗ NodeSelector / Affinity rules match no nodes.
- ⊗ Taints exist, but Pod has no matching toleration.
- ⊗ Topology spread constraints cannot be satisfied.
- ⊗ Insufficient PersistentVolume / StorageClass.
- ⊗ HostPort / NodePort already in use.
- ⊗ Node has max Pods reached.

DEBUGGING STEPS

1. `kubectl describe pod <pod>` → Check Events section
2. Check node capacity: `kubectl top nodes`
3. Check taints: `kubectl describe node <node>`
4. Verify labels: `kubectl get nodes --show-labels`
5. Review scheduler messages in Events.

AFFINITY EXAMPLE (REQUIRED)

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values: ["ssd"]
```

ANTI-AFFINITY EXAMPLE

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      labelSelector:
        matchLabels:
          app: myapp
      topologyKey: kubernetes.io/hostname
```

SENIOR ENGINEER TIP

The scheduler is predictable.
Give it good requests, clear rules, and correct labels.
Bad requests + bad rules = unschedulable Pods and unhappy users.

KEY REMINDER

Scheduler makes decisions only at Pod creation time.
After that, the Kubelet keeps the Pod running.

10. HEALTH CHECKS AND APPLICATION AVAILABILITY

THE 3 TYPES OF PROBES

1. LIVENESS PROBE

- Checks if the application is alive.
- If it fails, the container is restarted.
- Prevents stuck or dead processes.

Use for: Detecting failures during runtime.

2. READINESS PROBE

- Checks if the application is ready to serve traffic.
- If it fails, Pod is removed from Service endpoints.
- No restart happens.

Use for: Traffic gating and rolling updates.

3. STARTUP PROBE

- Checks if the application has started successfully.
- Disables liveness/readiness until startup succeeds.
- Helps slow-start apps avoid false restarts.

Use for: Apps with long startup time.

PROBE FAILURE BEHAVIOR

PROBE	ON FAILURE
Liveness	Container is restarted according to restart policy.
Readiness	Pod is marked Unready and removed from Service endpoints.
Startup	Liveness and Readiness probes stay inactive until it succeeds.

SERVICE ENDPOINT REMOVAL (READINESS)

Pod Becomes Unready → Removed from Service Endpoints → No Traffic Sent to This Pod

- When readiness fails, Kubernetes protects your users by stopping traffic.

GRACEFUL STARTUP

1. Container starts
2. Startup probe checks
3. Once startup succeeds:
   - Liveness and Readiness start
4. Pod becomes Ready and receives traffic

Prevents premature restarts and failed probes during warm-up.

GRACEFUL SHUTDOWN

1. Pod marked for deletion
2. PreStop hook (if any)
3. SIGTERM sent to container
4. Application shuts down cleanly
5. Pod terminated after grace period

Allows in-flight requests to finish safely.

PRODUCTION FAILURE: LIVENESS PROBE CREATES A RESTART LOOP

WHAT HAPPENS?

- Liveness probe is too strict.
- App takes longer to respond.
- Probe fails → Container restarts.
- App never gets stable → Loop continues.

HOW TO PREVENT

- Set correct initialDelaySeconds.
- Use Startup Probe for slow apps.
- Increase timeouts and failureThreshold.
- Monitor and test under load.

QUICK REFERENCE (DEFAULTS)

- initialDelaySeconds: 0
- periodSeconds: 10
- timeoutSeconds: 1
- failureThreshold: 3
- successThreshold (Readiness): 1

SENIOR ENGINEER TIP

Probes are about protecting users, not checking metrics.
Liveness restarts the app.
Readiness protects the user.
Startup prevents premature failure.

11. PERSISTENT STORAGE

THE STORAGE BUILDING BLOCKS

POD — Uses storage → **PVC** — Requests storage → **PV** — Provides storage → **STORAGECLASS** — Defines how storage is provisioned → **CSI DRIVER** — Talks to the storage system

VOLUME

A Volume is storage accessible to containers in a Pod.

- Types: emptyDir, hostPath, configMap, secret, persistentVolumeClaim, ...
- Persistent data uses PV + PVC.

PV vs PVC

PV (PERSISTENT VOLUME)

- Cluster resource
- Actual storage in the infra
- Created by admin or dynamically
- Lifecycle independent of Pods

↔

PVC (PERSISTENT VOLUME CLAIM)

- Namespaced resource
- Request for storage
- Consumes a PV
- Lifecycle tied to the claim

DYNAMIC PROVISIONING FLOW

PVC Created → StorageClass Selected → CSI Driver Provisions → PV Created → Bound to PVC

You usually don't create PVs manually anymore. StorageClass + CSI handle it for you.

STORAGECLASS

- Describes the storage type and parameters.
- Tells Kubernetes how to provision storage.
- Supports multiple storage tiers (fast, standard, cheap).
- Can set default StorageClass.

```
Example:
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  fsType: ext4
reclaimPolicy: Delete
```

ACCESS MODES

MODE	DESCRIPTION
RWO	ReadWriteOnce Mounted read-write by a single node.
ROX	ReadOnlyMany Mounted read-only by many nodes.
RWX	ReadWriteMany Mounted read-write by many nodes.
RWOP	ReadWriteOncePod Read-write by one Pod (no other Pods).

RECLAIM POLICIES

POLICY	BEHAVIOR
Delete	Delete the underlying storage when PV is released.
Retain	Keep the storage. PV moves to Released state.
Recycle (Deprecated)	Kubernetes runs a basic clean-up (rm -rf /*). Not recommended.

For production: use Retain for important data. Delete only for disposable data.

STATEFUL STORAGE FAILURE PATH

1. **POD CRASHES or NODE FAILS** — Pod goes down.
2. **POD RESTARTS on another node** — Kubelet starts containers.
3. **POD ATTACHES PVC** — Kubelet requests volume.
4. **VOLUME ATTACH FAILS** — Reasons: disk issue, permissions, network, IO limits, etc.
5. **POD STUCK in PENDING or CRASH LOOP** — App unavailable.
6. **MANUAL ACTION REQUIRED** — Fix storage issue, reattach or provision new PV.

⚠ Always monitor: node status, volume attach/detach events, storage errors, CSI driver logs.

BEST PRACTICES

- Use StorageClass and dynamic provisioning.
- Set correct access modes.
- Choose the right reclaim policy.
- Monitor storage metrics and events.

COMMON COMMANDS

```
kubectl get pvc,pv,sc
kubectl describe pvc <name>
kubectl get events --sort-by=.lastTimestamp
kubectl logs -n kube-system -l app=csi-driver
```

KEY TAKEAWAY

PV = Storage in the cluster
PVC = Request for storage
StorageClass + CSI = Automation
Good storage design = Reliable apps

PRODUCTION NOTE

Storage is the most common reason for outages in stateful apps.
Design it with redundancy, monitoring and clear recovery steps.

REMEMBER

Data is precious.
Back it up. Protect it.
Test your recovery.

12. STATEFULSETS, DAEMONSETS & JOBS

WHEN DEPLOYMENT IS WRONG

- X You need stable identities
- X You need stable storage
- X You need one Pod per node
- X You need a task, not a service
- X You need a scheduled task

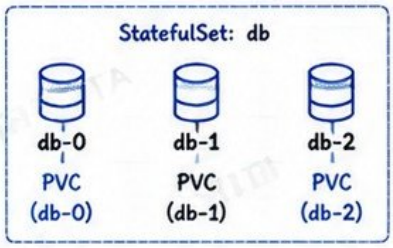
Use the RIGHT controller for the job.

CONTROLLER OVERVIEW

DEPLOYMENT	STATEFULSET	DAEMONSET	JOB	CRONJOB
Stateless apps Scale & update replicated Pods	Stable identity Stable storage Ordered ops	One Pod per node Node-level workloads	Run to completion Then stop	Run Jobs on a schedule (repeatedly)

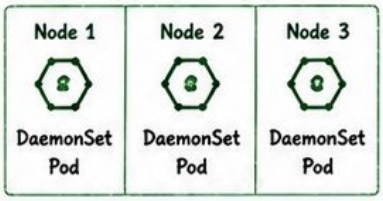
STATEFULSET IN A NUTSHELL

- Each Pod gets a STABLE ID (ordinal).
- Pods are created in ORDER (0 → N).
- Stable network identity: <name>-<ordinal>
- Stable storage via PVC per Pod.
- Ideal for databases, message brokers, and systems with local state.



DAEMONSET IN A NUTSHELL

- Runs exactly ONE Pod on each (or selected) node.
- Automatically adds Pod on new nodes.
- Great for node-level tasks.
- Examples: logging, monitoring, node exporters, CNI plugins.



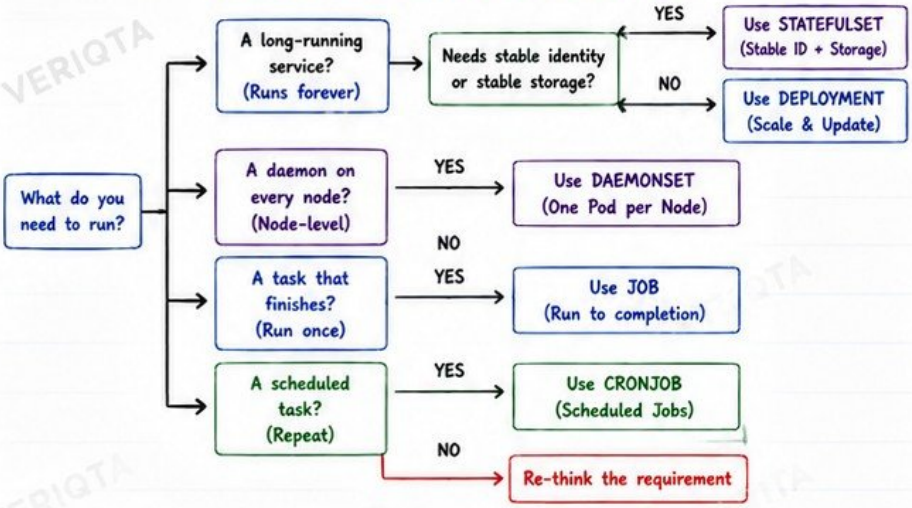
JOB IN A NUTSHELL

- Runs a Pod until the task completes successfully.
- Retries on failure (if configured).
- Does not run forever.
- Good for batch tasks, migrations, backups, data processing.

CRONJOB IN A NUTSHELL

- Creates Jobs on a schedule.
- Each Job runs to completion.
- Supports concurrency and history limits.
- Perfect for backups, reports, cleanups.

WORKLOAD SELECTION DECISION TREE



PRODUCTION EXAMPLES

DEPLOYMENT
Web apps, APIs, microservices, stateless backends, workers.

STATEFULSET
MongoDB, MySQL, PostgreSQL, Kafka, Redis, Elasticsearch.

DAEMONSET
node-exporter, fluentd, log collectors, CNI, security agents.

JOB
Database migration, data import, one-time setup, backups.

CRONJOB
Daily backups, cleanup jobs, report generation, notifications.

PRODUCTION NOTE

- Wrong controller choice = pain later (data loss, complex migrations).
- Match the controller to the WORKLOAD'S NATURE, not convenience.
- Think: Identity? Storage? Node? Duration? Schedule?

SENIOR TIP

When in doubt, design for failure.
Use the controller that gives you correct behavior under failure.
That's what keeps you calm at 3AM.

13. RBAC AND KUBERNETES SECURITY

AUTHENTICATION vs AUTHORIZATION

AUTHENTICATION
(Who are you?)

- Verifies identity.
- Examples: Users, ServiceAccounts, OIDC, Tokens, Client Certs.

→

AUTHORIZATION
(What can you do?)

- Controls access.
- Managed by RBAC (Roles, Bindings, Permissions).

RBAC BUILDING BLOCKS

SERVICEACCOUNT	Identity for Pods. Used by workloads to access the API.
ROLE	Defines permissions within a NAMESPACE.
CLUSTERROLE	Defines permissions across the CLUSTER.
ROLEBINDING	Grants a Role to a User, Group or ServiceAccount in a Namespace.
CLUSTERROLEBINDING	Grants a ClusterRole to a User, Group or ServiceAccount cluster-wide.

HOW RBAC WORKS

User / ServiceAccount → is bound to → Role or ClusterRole → which grants access to → Resources (API Objects)

Examples: pods, secrets, configmaps, deployments, nodes, namespaces, etc.

LEAST PRIVILEGE PRINCIPLE

- Give only the permissions needed.
- Avoid wildcard (*) permissions.
- Apply at the narrowest scope (Namespace > Cluster).
- Review and audit regularly.

LESS ACCESS = LESS RISK

kUBECTL AUTH CAN-I

Check what a user or ServiceAccount is allowed to do.

```
# Examples
kubectl auth can-i get pods
kubectl auth can-i get secrets -n prod
kubectl auth can-i create deployments
 --as=system:serviceaccount:prod:app-sa
kubectl auth can-i --list
```

SERVICEACCOUNT IN A POD

POD

ServiceAccount: app-sa

↓

Used by containers to access the API

↓

Bound through RBAC

COMMON RBAC BINDING PATHS

NAMESPACE-LEVEL ACCESS

User / Group / SA + ROLE + ROLEBINDING → Resources in Namespace

CLUSTER-LEVEL ACCESS

User / Group / SA + CLUSTERROLE + CLUSTERROLEBINDING → Cluster Resources

SECURITY REMINDER

Avoid unnecessary cluster-admin.
It has FULL ACCESS to everything in the cluster.
Use least privilege. Review access often.

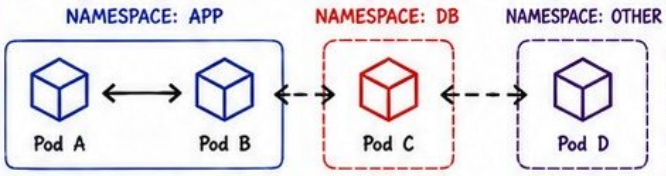
cluster-admin = full control (Use Carefully!)

QUICK REFERENCE

- Authentication verifies identity.
- Authorization controls access.
- RBAC manages authorization.
- Least privilege keeps you safe.
- kubectl auth can-i verifies permissions.

14. NETWORKPOLICY AND WORKLOAD ISOLATION

DEFAULT KUBERNETES CONNECTIVITY



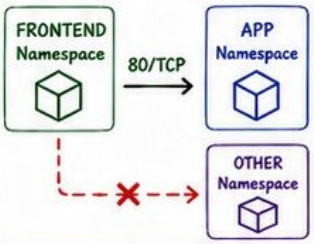
- By default, all Pods can communicate with every other Pod in the cluster.
- This is ALLOW ALL traffic.

NETWORKPOLICY IN A NUTSHELL

- Controls traffic at Layer 3 (IP) and Layer 4 (Port).
- Applied to Pods.
- Policies are additive within the same direction.
- If no policy selects a Pod, it allows all traffic.
- If a policy selects a Pod, default deny applies unless allowed by rules.

INGRESS POLICIES

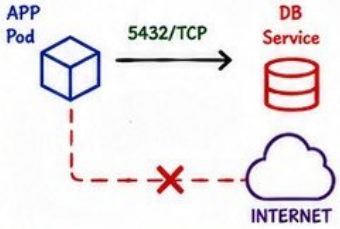
Example: Allow only frontend Pods to access app Pods on port 80



- Controls incoming traffic to selected Pods.
- You define who can talk to the Pod and on which ports.
- If a Pod is selected and no rule allows traffic, everything is denied.

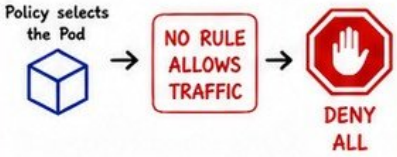
EGRESS POLICIES

Example: Allow app Pods to access only DB service on port 5432



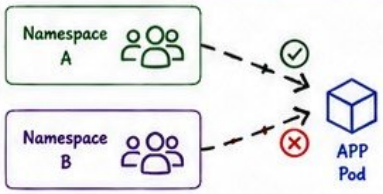
- Controls outgoing traffic from selected Pods.
- You define where the Pod is allowed to connect.
- If a Pod is selected and no rule allows traffic, everything is denied.

DEFAULT DENY MODEL



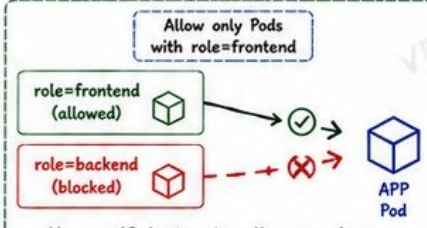
Once a policy selects a Pod, default behavior is DENY ALL.

NAMESPACE SELECTORS



Use namespaceSelector to allow or deny traffic from entire namespaces.

POD SELECTORS



Use podSelector to allow or deny traffic from specific Pods.

CNI ENFORCEMENT

- NetworkPolicy is just an API object.
- Your CNI plugin enforces the rules on the nodes.
- Popular CNI plugins with NetworkPolicy support:

If your CNI doesn't support NetworkPolicy, policies won't have any effect.

ZERO-TRUST WORKLOAD COMMUNICATION

- Never trust implicit network access.
- Explicitly allow only what is needed.
- Apply default deny everywhere.
- Monitor and audit your policies.
- Combine with RBAC, Secrets, and least privilege.

SECURITY REMINDER

NetworkPolicy is a critical layer of defense.
Don't rely on default allow.
Assume breach. Enforce least network access.

Default allow = High risk

Default is ALLOW ALL.

Once selected, default is DENY.

Use Ingress to control incoming traffic.

Use Egress to control outgoing traffic.

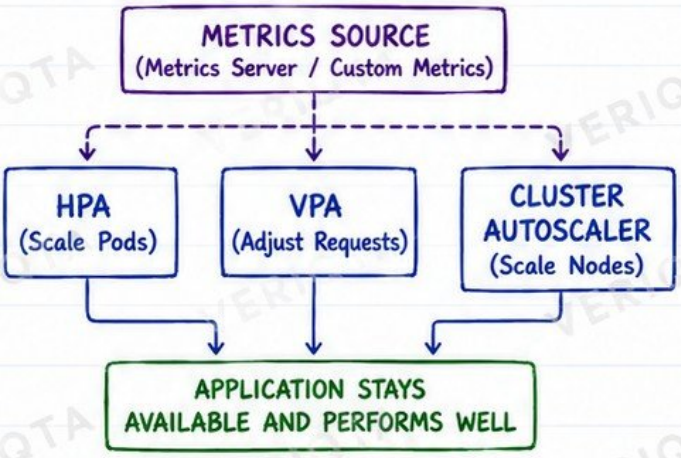
Policies apply to Pods.

15. AUTOSCALING AND CAPACITY

THE AUTOSCALING TOOLKIT

HPA	**Horizontal Pod Autoscaler** Adjusts the number of Pods based on metrics.
VPA	**Vertical Pod Autoscaler** Adjusts CPU/Memory requests for containers.
CLUSTER AUTOSCALER	Adds or removes nodes based on pending Pods and resource demand.

HOW IT WORKS TOGETHER

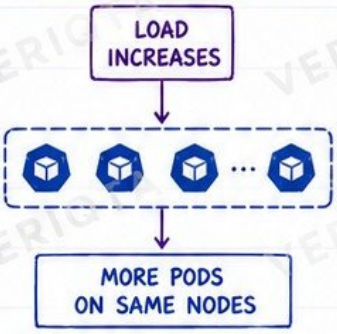


METRICS YOU CAN USE

	CPU METRICS cpu_usage cpu_utilization
	MEMORY METRICS memory_usage_bytes memory_utilization
	CUSTOM METRICS requests_per_second queue_depth error_rate latency_p95

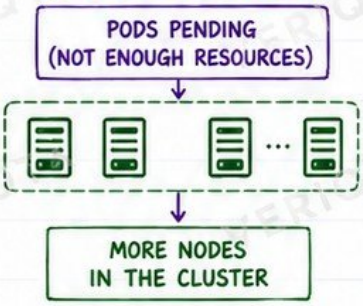
SCALING PODS VS SCALING NODES

SCALING PODS (HPA)



- Faster response
- Good for stateless apps
- Limited by node capacity

SCALING NODES (CLUSTER AUTOSCALER)



- Adds more capacity
- Needed for sustained growth
- Takes minutes to provision

METRICS SERVER

- Collects resource usage from kubelets
- Stores in memory (short term)
- Used by HPA, VPA and other components
- Does NOT store historical data
- Install in every cluster
- Command: kubectl top nodes / pods

SCALING NOTE (CRITICAL!)

AUTOSCALING CANNOT REPAIR BAD RESOURCE REQUESTS.

If requests are too low or too high:

- Scheduling will be wrong
- Scaling will react late or overreact
- Performance and cost will suffer

KEY TAKEAWAYS

HPA = SCALE PODS
Handles traffic and demand changes.

VPA = RIGHT SIZE
Keeps resource requests realistic.

CLUSTER AUTOSCALER
Adds nodes when needed.

GOOD METRICS
Accurate metrics make scaling effective.

RIGHT SIZE FIRST
Then autoscaling delivers the best results.

☆ OBSERVE → MEASURE → RIGHT SIZE → SCALE → STAY RELIABLE

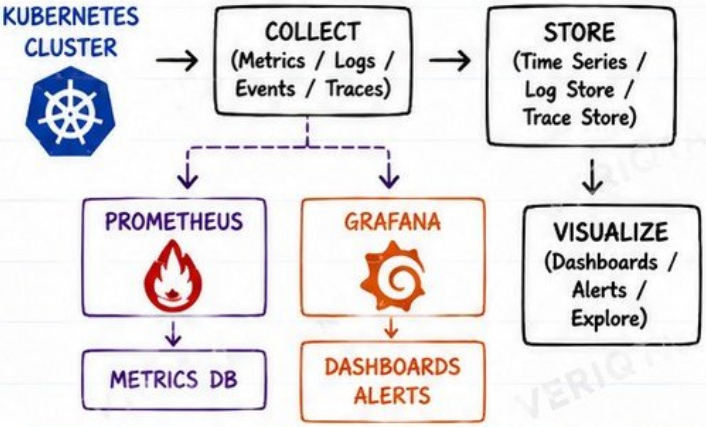
16. KUBERNETES OBSERVABILITY

YOU CAN'T IMPROVE WHAT YOU CAN'T OBSERVE

1. THE FOUR PILLARS OF OBSERVABILITY

METRICS	LOGS	EVENTS	TRACES
Numerical measurements over time.	Discrete records of what happened.	Important state changes in Kubernetes.	Request flow across services.
Answer: HOW MUCH?	Answer: WHAT HAPPENED?	Answer: WHAT CHANGED?	Answer: WHY SLOW?

2. CORE OBSERVABILITY STACK



3. KUBERNETES EVENTS

- Generated by the control plane and components
- Show state changes and reasons
- Very helpful for real-time troubleshooting
- Example: FailedScheduling, PullBackOff, OOMKilled, Unhealthy

```
kubectl get events -A --sort-by=.metadata.creationTimestamp
kubectl describe pod <pod-name>
```

4. APPLICATION VS INFRASTRUCTURE SIGNALS

INFRASTRUCTURE SIGNALS

- Node CPU / Memory
- Pod CPU / Memory
- Disk / Network
- Kubelet / API Server
- Controller Manager
- etcd health
- Scheduler

APPLICATION SIGNALS

- Request rate
- Error rate
- Latency (p95/p99)
- Throughput
- Queue depth
- Business metrics
- User experience

5. GOLDEN SIGNALS (USE THESE FIRST)

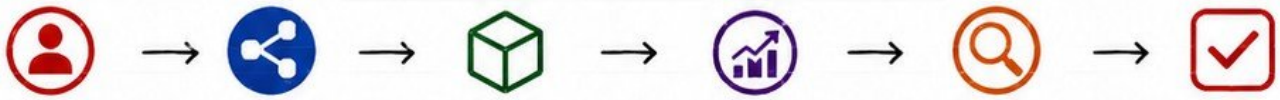
LATENCY
How long requests take (p50/p95/p99). High latency = slow experience.

TRAFFIC
How many requests are being handled. Traffic changes drive load.

ERROR RATE
The rate of failed requests. High errors = unreliable system.

SATURATION
How full your resources are. High saturation = risk of failure.

6. PRODUCTION TROUBLESHOOTING SIGNAL FLOW



PRODUCTION NOTE

Observability is not just dashboards.
It is having the right signals, in the right place, at the right time, to quickly understand system behavior and user experience.

REMEMBER:

- Instrument your application
- Monitor your infrastructure
- Correlate everything
- Act before users complain

GOOD OBSERVABILITY = FAST RECOVERY

17. KUBERNETES TROUBLESHOOTING WORKFLOW

THE RIGHT COMMANDS. IN THE RIGHT ORDER.

1. ESSENTIAL KUBECTL COMMANDS

kubectl get	Quick overview of resources and status.
kubectl describe	Deep details, events, conditions, and reasons.
kubectl logs	View logs from containers in a Pod.
kubectl exec	Run a command inside a running container.
kubectl events	Real-time events happening in the cluster.
kubectl top	CPU and memory usage for Nodes and Pods.
kubectl debug	Debug a running Pod or Node (ephemeral container).

2. INVESTIGATION PATH (FOLLOW THIS ORDER)

1. POD
- Is the Pod running?
- Check status, restarts, events and logs.

↓

2. SERVICE
- Is the Service selecting the right Pods?
- Check Endpoints.

↓

3. DNS
- Can the Pod resolve the name?
- Check CoreDNS and DNS records.

↓

4. NETWORK
- Is traffic allowed?
- Check NetworkPolicy, connectivity, ports.

↓

5. NODE
- Is the Node healthy?
- Check resources, pressure, kubelet and runtime.

3. QUICK WORKFLOW EXAMPLE

POD NOT WORKING → CHECK POD STATUS & EVENTS (kubectl get pods, kubectl describe pod) → CHECK LOGS (kubectl logs <pod-name>) → TEST INSIDE POD (kubectl exec -it <pod> -- /bin/sh) → CHECK SERVICE, DNS, NETWORK (kubectl get svc, ep nslookup <svc>) → CHECK NODE HEALTH (kubectl get nodes, kubectl describe node)

4. WHAT TO LOOK FOR AT EACH LAYER

POD
- Pending
- CrashLoopBackOff
- ImagePullBackOff
- OOMKilled
- Readiness/Liveness probes

SERVICE
- No Endpoints
- Wrong selector
- Port mismatch
- Session affinity issues

DNS
- Name not resolving
- Incorrect namespace
- CoreDNS errors
- Search domain / ndots issues

NETWORK
- NetworkPolicy blocking
- Wrong port / protocol
- Connectivity timeout
- MTU / CNI problems (rare)

NODE
- NotReady
- Resource pressure
- Disk / Memory / PID issues
- Kubelet down

DEBUGGING INSIGHT

Always troubleshoot FROM SYMPTOMS TOWARD DEPENDENCIES. Start where the problem appears, then move DOWNSTREAM through the system until you find the root cause.

GOLDEN RULE

Check the simplest thing first. Don't guess. Observe, verify, and move step by step.

OBSERVE → UNDERSTAND → ISOLATE → FIX → VALIDATE

18. NODE FAILURES AND POD EVICTION

1. NODE CONDITIONS

CONDITION	MEANING
MemoryPressure	Node is under memory pressure. Pods may be evicted.
DiskPressure	Low disk space. New Pods may not be scheduled.
PIDPressure	Node is running out of process IDs.
Node NotReady	Kubelet is not reporting. Node may be down or unreachable.

Check: `kubectl get nodes`
`kubectl describe node <node-name>`

2. EVICTIONS (HOW K8S PROTECTS THE NODE)

WHEN THE KUBELET EVICTS PODS

- MemoryPressure
- DiskPressure
- PIDPressure
- Node is low on resources

EVICTION ORDER

1. BestEffort Pods
2. Burstable Pods (low priority)
3. Burstable Pods (higher priority)
4. Guaranteed Pods (last resort)

Goal: keep the node healthy and kubelet running.

3. POD DISRUPTION & PODDISRUPTIONBUDGET

POD DISRUPTION

Any voluntary action that may cause Pods to go down:

- Node drain
- Node maintenance
- Cluster upgrades
- Descaling
- Manual deletion

PODDISRUPTIONBUDGET (PDB)

Ensures a minimum number of Pods stay available.

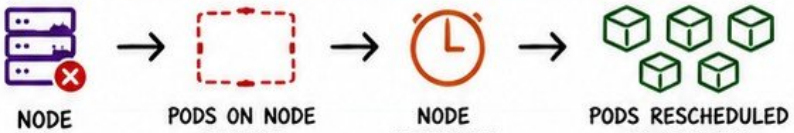
Example:

- minAvailable: 2 (at least 2 Pods must always be up)
- maxUnavailable: 1 (only 1 Pod can be down at a time)

4. DRAINING NODES (SAFE WAY)

1. Cordon the node (stop new Pods)
   `kubectl cordon <node-name>`
2. Drain the node (evict Pods safely)
   `kubectl drain <node-name> \`
   `--ignore-daemonsets \`
   `--delete-emptydir-data`
3. Pods move to other nodes
   Node is safe to maintain.
4. Do maintenance / upgrade
5. Uncordon the node (allow scheduling)
   `kubectl uncordon <node-name>`

5. WHAT HAPPENS DURING A NODE FAILURE?



NOTES

- Pods are NOT lost. They are rescheduled.
- Stateful workloads rely on PDB + topology + storage.
- Recovery time depends on cluster capacity.

PRODUCTION FAILURE: LOSING CAPACITY DURING MAINTENANCE

If you drain too many nodes at once WITHOUT PDBs or enough spare capacity:

- Too many Pods become unavailable
- Services degrade or become unavailable
- Users experience downtime
- Recovery takes longer

AVOID THIS BY:

- ✓ Use PDBs for critical workloads
- ✓ Maintain enough spare capacity (20-30% headroom)
- ✓ Drain one node at a time
- ✓ Monitor during maintenance
- ✓ Test failure scenarios regularly

KEY MINDSET: NODES FAIL. PLAN FOR IT. DESIGN FOR RESILIENCE.

19. KUBERNETES PRODUCTION RELIABILITY

BUILD FOR FAILURES. DESIGN FOR RECOVERY. OPERATE WITH CONFIDENCE.

1. MULTI-NODE ARCHITECTURE

- Never run on a single node.
- Distribute Pods across multiple nodes.

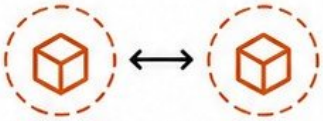
2. MULTI-AZ PLACEMENT

- Spread your workloads across multiple Availability Zones.
- Survive zone failures.

3. REPLICA STRATEGY

- Run multiple replicas for stateless workloads.
- Set sensible replica counts for availability and capacity.

4. POD ANTI-AFFINITY



- Keep replicas off the same node.
- Reduce risk of single node failure taking out all replicas.

5. TOPOLOGY SPREAD

- Use topologySpreadConstraints.
- Spread Pods evenly across zones and nodes.

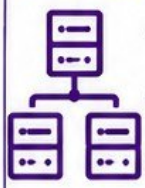
6. POD DISRUPTION BUDGETS (PDBs)

- Limit voluntary disruptions.
- Ensure minimum number of Pods stay available during updates and maintenance.

7. RESOURCE HEADROOM

- Don't run at 100% capacity.
- Keep 20–30% headroom.
- Allows for spikes, failures and scaling.

8. CONTROL-PLANE RELIABILITY



- Run control plane with multiple replicas.
- Use stable, supported Kubernetes versions.
- Monitor and alert on control-plane health.

9. BACKUP AND RESTORE

- Backup etcd regularly.
- Store backups securely off-cluster.
- Test restore process periodically.

Assume something WILL fail.

Design so your system keeps serving traffic.

10. FAILURE-DONG

✓ NODE FAILURE	→	Pods reschedule to healthy nodes.
✓ ZONE FAILURE	→	Workloads continue in other zones.
✓ NETWORK PARTITION	→	System degrades gracefully and recovers.
✓ CONTROL-PLANE FAILURE	→	Highly available control plane keeps the cluster running.

RELIABILITY IS A SYSTEM, NOT A SINGLE SETTING

1. Distribute across nodes and zones
2. Run enough replicas
3. Protect with PDBs
4. Keep resources healthy
5. Backup everything that matters
6. Plan for failure, not just success

PRODUCTION FAILURE: LOSING CAPACITY DURING MAINTENANCE

- ✗ Draining too many nodes at once
- ✗ No PDBs or too low minAvailable
- ✗ All replicas on the same node/zone
- ✗ Running at 100% capacity
- ✗ No backups when disaster strikes

PREVENT IT. PLAN IT. TEST IT.

☆ DESIGN FOR RESILIENCE TODAY. SLEEP WELL TOMORROW. ☆

20. KUBERNETES PRODUCTION MENTAL MODEL

ALL THE PIECES. ONE PICTURE.

USER
Client / Browser Application

→ **DNS**
Resolves domain to Load Balancer

→ **LOAD BALANCER**
Public entry point (Cloud LB or On-Prem LB)

→ **INGRESS / GATEWAY**
Routes traffic (host, path, TLS)

→ **SERVICE**
Stable endpoint and load balancing

→ **POD**
Runs your application container(s)

CONFIG / SECRET
- ConfigMap
- Secret
- Environment
- Mounted files

PVC → STORAGE
- PersistentVolume
- PersistentVolumeClaim
- StorageClass
- CSI (e.g., EBS, EFS, Ceph)

CONTROL PLANE

API SERVER
Validates requests and manages the cluster

ETCD
Stores cluster state (key-value store)

SCHEDULER
Decides which node runs the Pod

CONTROLLER MANAGER
Runs controllers to maintain desired state

WORKER NODES

Node 1	Node 2	Node 3
kubelet Manages Pods on this node	**kubelet** Manages Pods on this node	**kubelet** Manages Pods on this node
kube-proxy Handles Service networking	**kube-proxy** Handles Service networking	**kube-proxy** Handles Service networking
Container Runtime Runs containers (e.g., containerd)	**Container Runtime** Runs containers (e.g., containerd)	**Container Runtime** Runs containers (e.g., containerd)

CONTROLLERS

Reconcile actual state to desired state.

Examples: Deployment, StatefulSet, DaemonSet, Job, CronJob.

OBSERVABILITY

Metrics, logs, events and traces.

Used for detection, troubleshooting and performance analysis.

Common tools: Prometheus, Grafana, Loki, OpenTelemetry.

AUTOSCALING

Responds to demand.

- HPA – scale Pods
- VPA – adjust requests
- Cluster Autoscaler – scale nodes

Uses metrics from Metrics Server or custom metrics.

SECURITY

Protects your cluster and workloads.

- RBAC (access control)
- NetworkPolicy (traffic control)
- ServiceAccounts
- Secrets management
- Least privilege

NETWORKING

Enables communication between components.

- Services
- Ingress / Gateway
- DNS (CoreDNS)
- CNI Plugin
- NetworkPolicies

FINAL STAFF ENGINEER CHECKLIST

- ☑ Availability – multi-AZ, replicas, PDBs, no single points of failure
- ☑ Capacity – right requests/limits, autoscaling, headroom
- ☑ Security – RBAC, NetworkPolicy, secrets, image security
- ☑ Observability – metrics, logs, traces, alerts, dashboards
- ☑ Failure recovery – backup/restore, tested runbooks, disaster recovery
- ☑ Operability – automation, documentation, monitoring, on-call readiness

PRODUCTION MENTAL MODEL

"Kubernetes is a control system for running, scaling and operating distributed applications reliably, not just a container orchestrator."

KEY TAKEAWAY

Understand how all the components work together, so you can design, operate and troubleshoot Kubernetes at production scale.

PLAN → DEPLOY → OBSERVE → SCALE → SECURE → OPERATE